PBL · Fase 5 · Melhores Compras

Análise de Dados para Ciência de Dados

Segundo desafio — Explorando o arquivo vendas_linha_petshop_2020_2024.csv

Grupo: Paladinos · Turma 2TSCOA

Integrantes:

RM 566173 — Igor Cardoso Soares da Silva

RM 562692 — Murilo Jucá de Carvalho

RM 561821 — Rozana da Malta Martins

RM 566167 — Ruan Duarte Garcia

Contexto e preparação

A análise foi feita em **Python (pandas)**. O arquivo tem **250.059 linhas e 14 colunas**, e o original não foi alterado. Antes de responder às perguntas, os dados foram tratados (script `limpeza.py`): leitura em Latin-1 com separador ";", conversão de números em formato brasileiro e "R$", reconstrução de 324 quantidades vazias e descarte de 441 registros inválidos (negativos, zero ou sem conserto). Restaram **249.618 registros válidos**.

```
import pandas as pd
df = pd.read_csv('vendas_linha_petshop_2020_2024.csv', sep=';', encoding='latin-1',
dtype=str)
print(df.shape)            # (250059, 14)
print(df.isna().sum())     # quantidade: 332 nulos (+147 em branco, 'teste', 'qtd x')
```

1) Estratégia para bases de treino, teste e validação

Os registros são **vendas ao longo do tempo**, e qualquer modelo preditivo útil (ex.: prever receita ou quantidade) será usado para **prever o futuro**. Por isso, a estratégia recomendada é uma **divisão temporal (holdout cronológico)**, e não um sorteio aleatório. Sortear linhas colocaria vendas de 2024 no treino e de 2021 no teste: o modelo "veria o futuro" (vazamento de dados), e o desempenho medido seria otimista demais.

Base	Período	Registros	Proporção	Justificativa
Treino	jan/2020 – dez/2022	150.370	60%	Maior parte dos dados, com 3 ciclos anuais completos para o modelo aprender tendência e sazonalidade (fev/out/dez).
Validação	jan/2023 – dez/2023	50.081	20%	Ano completo e posterior ao treino, usado para ajustar hiperparâmetros e escolher o modelo sem tocar no teste.
Teste	jan/2024 – abr/2024	49.167	20%	Dados mais recentes, usados uma única vez no fim para medir o desempenho real em período "futuro".

Por que 60/20/20 e não 70/15/15 ou 80/10/10?

- A divisão coincide com **anos fechados**, então cada base contém um ciclo sazonal inteiro (exceto 2024, que é parcial por natureza). Um corte em 70% cairia no meio de 2022 e deixaria a validação sem um ano completo.
- Com ~250 mil linhas, **20% ainda são ~50 mil registros** em validação e em teste, amostra mais que suficiente para métricas estáveis. Não é preciso concentrar tudo no treino.
- A **validação separada do teste** evita escolher o modelo "olhando" o teste. O teste continua sendo uma estimativa honesta.

Cuidados complementares:

- Dentro do treino, usar **validação cruzada temporal** (`TimeSeriesSplit`), em que cada dobra treina no passado e valida no período seguinte.
- **Calcular limites de outliers, médias de imputação e normalizações só no treino** e aplicá-los depois em validação e teste, sem recalcular, para evitar vazamento.
- **Monitorar mudança de distribuição (drift):** 2022 e 2024 vieram em outro formato de exportação e 2024 tem volume diário ≈3,7× maior. Vale comparar as distribuições das bases antes de treinar.

- Para um problema **sem dependência temporal** (ex.: classificar a categoria de um produto), a alternativa seria **70/15/15 aleatório estratificado** (por categoria/região), com `random_state` fixo.

```
treino    = df[df['data'] <  '2023-01-01']
validacao = df[(df['data'] >= '2023-01-01') & (df['data'] < '2024-01-01')]
teste     = df[df['data'] >= '2024-01-01']

from sklearn.model_selection import TimeSeriesSplit   # validação cruzada dentro do treino
tscv = TimeSeriesSplit(n_splits=5)
```

2) Variáveis categóricas e numéricas

Variável numérica (quantitativa) representa uma **medida ou contagem** sobre a qual operações matemáticas fazem sentido (somar, tirar média). Ela pode ser:

- **discreta:** valores inteiros e contáveis (ex.: quantidade de itens);
- **contínua:** qualquer valor num intervalo (ex.: valor em reais).

Variável categórica (qualitativa) representa **grupos, rótulos ou classes**. Não faz sentido calcular média, mesmo quando o valor está escrito com números. Ela pode ser:

- **nominal:** sem ordem (ex.: estado, forma de pagamento);
- **ordinal:** com ordem natural (ex.: mês, porte P/M/G).

Na modelagem, variáveis numéricas entram diretamente (às vezes normalizadas). Categóricas precisam ser **codificadas** (one-hot/dummies, target encoding etc.).

Atenção ao arquivo: ao ler o CSV, o pandas trata `valor`, `quantidade`, `valor_total_bruto`, `valor_comissao` e `lucro_liquido` como **texto (object)**, por causa da vírgula decimal, do "R$" e de conteúdos como "teste". Essas colunas são **numéricas por natureza** e precisam de conversão antes da análise.

Coluna	Tipo	Subtipo	Observação na base
cod_pedido	Categórica	Identificador (nominal)	Escrito com números, mas é um rótulo; não faz sentido somar ou tirar média
regiao_pais	Categórica	Nominal	5 regiões
produto	Categórica	Nominal	24 produtos distintos na amostra
valor	Numérica	Contínua	Preço unitário (R$)
quantidade	Numérica	Discreta	Itens por registro: 1 a 1.000, com 98,1% até 6
valor_total_bruto	Numérica	Contínua	Derivada: valor × quantidade
data	Temporal	Data (dd/mm/aaaa)	Gera variáveis: ano, mês (ordinal), dia da semana (categórica)
estado	Categórica	Nominal	25 estados
formapagto	Categórica	Nominal	5 formas (inclui "Dinheiro", fora do dicionário)
centro_distribuicao	Categórica	Nominal	5 centros
responsavelpedido	Categórica	Nominal	25 responsáveis
valor_comissao	Numérica	Contínua	R$ pago de comissão
lucro_liquido	Numérica	Contínua	R$ de lucro após impostos e despesas
categoriaprod	Categórica	Nominal	7 categorias

Resumo: numéricas são `valor`, `quantidade`, `valor_total_bruto`, `valor_comissao` e `lucro_liquido`. Categóricas são `cod_pedido`, `regiao_pais`, `produto`, `estado`, `formapagto`, `centro_distribuicao`, `responsavelpedido` e `categoriaprod`. `data` é temporal e dela se extraem atributos numéricos ou categóricos.

```
numericas = ['valor', 'quantidade', 'valor_total_bruto', 'valor_comissao', 'lucro_liquido']
categoricas = ['cod_pedido', 'regiao_pais', 'produto', 'estado', 'formapagto',
               'centro_distribuicao', 'responsavelpedido', 'categoriaprod']
df[categoricas].nunique()     # cardinalidade de cada categórica
```

3) Importância do código do pedido na análise preditiva

O `cod_pedido` é um **identificador**. Ele é essencial para **gerir os dados**, mas **não deve ser usado como variável preditiva**.

Onde ele é importante:

- **Rastreabilidade:** permite voltar ao pedido original para auditar uma previsão ou um registro suspeito (ex.: um outlier).
- **Deduplicação e integridade:** identifica registros repetidos e garante que um pedido não seja contado duas vezes.
- **Integração:** é a chave para cruzar com outras tabelas (clientes, entregas, devoluções, itens do pedido).
- **Separação correta das bases:** se um pedido tiver vários itens, todos devem ficar na mesma base (treino **ou** teste). Caso contrário, há vazamento.

Por que não usar como variável do modelo:

- **Não carrega informação de negócio:** o número é arbitrário. O pedido 150.581 não é "maior" nem "melhor" que o 50.284.
- **Gera correlações falsas:** o código é **sequencial no tempo** (2020: 50.284–100.372; 2024: 226.123–275.395). O modelo aprenderia "código alto = 2024" em vez do comportamento real, e a relação quebraria com os códigos novos.
- **Alta cardinalidade:** codificado como categoria, cria uma coluna por pedido e leva a overfitting. Tratado como número, cria uma ordem que não existe.

Achado na base: em 2023, **50.282 linhas compartilham 25.141 códigos** (cada código aparece duas vezes). Os pares **não são o mesmo pedido**: 0,0% têm o mesmo estado e só 0,3% a mesma data. Portanto, o código **não é uma chave única confiável** nessa base. Antes de modelar, é preciso corrigir a origem ou criar uma chave substituta (ex.: código + data + produto, ou um identificador sequencial novo).

```
df['cod_pedido'].duplicated().sum()                    # 25.141 códigos repetidos (todos em 2023)
X = df.drop(columns=['cod_pedido', 'lucro_liquido'])     # cod_pedido fora das variáveis do modelo
```

4) Quantidade de outliers e estratégia de identificação

Resultado: 5.364 registros identificados como outliers (2,1% dos 249.618 registros válidos), além de 441 registros **inválidos**. Os inválidos (quantidade negativa ou zero, campo vazio sem conserto) foram tratados antes, como **erro**, e não como outlier.

Estratégia utilizada — método do Intervalo Interquartil (IQR), aplicado por tipo de variável:

- Para cada variável, calculam-se o 1° quartil (Q1), o 3° quartil (Q3) e o IQR = Q3 − Q1. É outlier o valor **abaixo de Q1 − k·IQR ou acima de Q3 + k·IQR**.
- O IQR foi escolhido porque as variáveis são **assimétricas** (a maioria dos pedidos tem 1 a 3 itens, com cauda longa). Métodos baseados em média e desvio-padrão (z-score) são distorcidos pelos próprios

outliers. Como comparação, o z-score > 3 na quantidade encontraria 4.550 registros, um resultado próximo.

Regra	Como foi aplicada	Outliers
Quantidade	IQR 1,5 sobre toda a base: Q1 = 1, Q3 = 3, limite superior = 6 unidades	4.659
Valor unitário	IQR 1,5 **por produto** (cada item tem sua faixa de preço; um valor global marcaria todos os itens caros)	1
Lucro e comissão	Proporção sobre a receita, IQR 3,0 por produto, + regra de negócio: lucro líquido maior que a receita bruta é impossível (366 casos)	727
Total (sem dupla contagem)	22 registros aparecem em mais de uma regra	**5.364**

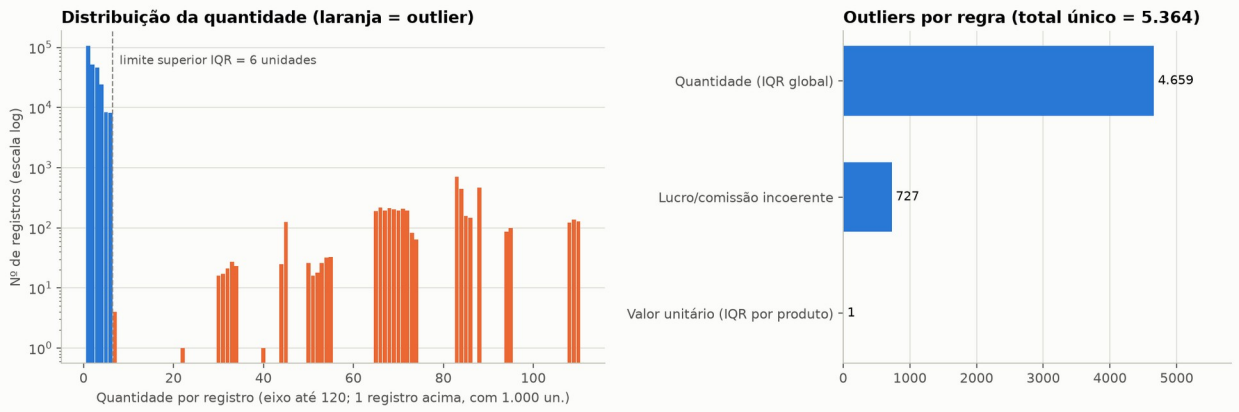


Figura 1 — À esquerda, distribuição da quantidade por registro (escala log) com o limite do IQR; à direita, outliers por regra.

O que os outliers revelam:

- **Quantidade:** não são valores soltos. **91% estão agrupados entre 65 e 110 unidades** (os mais frequentes são 83, 88 e 84), 410 ficam entre 7 e 64, e há 1 registro com 1.000 unidades (provável erro de digitação). Em 2020–2021 eles se concentram em fevereiro, outubro e dezembro; em 2023, em todos os meses. Representam só 1,9% dos registros, mas **46% da receita**. Parecem **vendas reais de atacado/B2B**, não erro.
- **Valor unitário:** 1 registro com preço fora do padrão do produto (ex.: Biscoito True Champion a R$ 2.500 em 2022), claramente erro de digitação.
- **Lucro/comissão:** registros com lucro acima da receita ou comissão desproporcional (ex.: Bandana Disney com lucro maior que o valor vendido). São inconsistências de cálculo.
- **Por ano:** 2020: 2.163; 2021: 1.476; 2022: 29; 2023: 1.677; 2024: 19.

```
def limites_iqr(s, k=1.5):
    q1, q3 = s.quantile([0.25, 0.75])
    return q1 - k * (q3 - q1), q3 + k * (q3 - q1)

lo, hi = limites_iqr(df['quantidade'])                          # (-2.0, 6.0)
out_qtd = (df['quantidade'] < lo) | (df['quantidade'] > hi)  # 4.659 registros

lim = df.groupby('produto')['valor'].apply(lambda s: pd.Series(limites_iqr(s), index=['lo',
'hi'])).unstack()
out_val = (df['valor'] < df['produto'].map(lim['lo'])) | (df['valor'] >
df['produto'].map(lim['hi']))

out_lucro = df['lucro_liquido'] > df['valor_total_bruto']   # + IQR 3,0 da proporção
lucro/receita e comissão/receita
outliers = out_qtd | out_val | out_lucro                        # 5.364 registros (com as
regras de proporção)
```

5) Melhor estratégia para tratar os outliers

Não existe um único tratamento correto. **A melhor estratégia depende da causa do outlier**: se é **erro** ou se é **comportamento real** que o modelo precisa aprender. Remover tudo apagaria 46% da receita; manter tudo sem critério ensinaria ao modelo dados errados. A proposta é tratar cada grupo de acordo com sua natureza:

Grupo	Diagnóstico	Tratamento recomendado
Pedidos de grande volume (4.659)	Vendas reais, agrupadas e sazonais (atacado/B2B)	**Manter**, exceto o registro de 1.000 unidades, tratado como erro de digitação (corrigir na origem ou remover). Criar a variável indicadora `grande_volume` ou modelar os segmentos separadamente (varejo × grande volume). Se o alvo for demanda de varejo, excluí-los desse modelo específico. Opcionalmente, usar escala logarítmica e modelos robustos (árvores, regressão de Huber).
Valor unitário fora da faixa (1)	Erro de digitação	**Corrigir** pelo preço mediano do produto na mesma data ou, se não houver referência, **remover** (impacto desprezível).
Lucro/comissão incoerente (727)	Erro de cálculo/cadastro	**Remover das análises e modelos de margem/lucro** (0,3% da base) ou recalcular pela margem mediana do produto. Para modelos de receita/quantidade, podem ser mantidos.
Registros inválidos (441)	Negativos, zero, texto sem conserto	**Remover**. Se houver regra de negócio para devoluções (quantidade negativa), tratá-las numa tabela própria.

Boas práticas no tratamento:

- **Não apagar do original:** marcar com flags (como feito em `vendas_tratadas.csv`). Assim cada decisão é reversível e auditável.
- **Calcular os limites só na base de treino** e aplicá-los em validação e teste, para não vazar informação do futuro.
- **Winsorização (capping)**, ou seja, limitar valores ao percentil 1%/99%, é útil quando o outlier é real mas extremo e o modelo é sensível (regressão linear). Não é recomendada aqui para quantidade, pois achataria justamente o segmento que traz metade da receita.
- **Comparar o modelo com e sem cada tratamento** na base de validação e manter a opção com menor erro.
- **Validar com a área de negócio:** confirmar se os pedidos de 65 a 110 unidades são mesmo atacado e corrigir na origem os erros de cálculo de lucro e comissão.